

Java Training

Chương 8

Collection

Collection là gì?

■ Collection là gì?

Là object thể hiện tập hợp của nhiều phần tử (object) với số phần tử không cố định.

■ Các loại collection

Loại	Giải thích	Class name
List	Quản lý đối tượng theo trình tự. Có thể lưu giữ phần tử trùng lặp.	ArrayList, LinkedList
Set	Quản lý đối tượng không theo trình tự. Lưu giữ phần tử đơn nhất	HashSet, TreeSet
Map	Quan hệ tương ứng giữa key và giá trị, quản lý object.	HashMap, TreeMap

ArrayList

■ ArrayList class

Là class để implement List interface, và thể hiện mảng có thể thay đổi size.

● Ví dụ

```
// Generate/sinh ArrayList quản lý data kiểu String
ArrayList<String> al = new ArrayList<String>();

// Add thêm phần tử vào cuối của ArrayList
al.add("Hi");    // "Hi"

// Lấy và hiển thị tất cả các phần tử của ArrayList
for (String str : al) {
    System.out.println(str);
}
```

ArrayList: Các method chính

Method	Giải thích
boolean add(E o)	Thêm phần tử đã được chỉ định vào cuối và trả về true.
void add(int index, E element)	Insert phần tử đã được chỉ định vào vị trí đã được chỉ định. Di chuyển phần tử hiện tại có ở vị trí đó và phần tử kế tiếp sang bên phải
E set(int index, E element)	Thay thế phần tử có ở vị trí đã được chỉ định bằng phần tử đã được chỉ định. Trả về phần tử đã có từ trước vào vị trí đã được chỉ định.
E get(int index)	Trả về phần tử có ở vị trí đã được chỉ định.
E remove(int index)	Delete phần tử khỏi vị trí đã được chỉ định. Di chuyển phần tử kế tiếp sang phía trái. Trả lại phần tử đã bị xóa.
int size()	Trả lại số phần tử.

ArrayList: Ví dụ

```
package chuong8;

import java.util.ArrayList;

public class ArrayList01 {
    public static void main(String[] args) {
        // Generate/sinh ArrayList để quản lý data kiểu String
        ArrayList<String> al = new ArrayList<String>();

        // Thêm phần tử vào cuối của ArrayList
        al.add("Hi"); // "Hi"
        al.add("Hello"); // "Hi" "Hello"

        // Thêm phần tử vào vị trí chỉ định của ArrayList
        al.add(0, "Java"); // "Java" "Hi" "Hello"

        // Thay thế phần tử của vị trí chỉ định của ArrayList
        al.set(1, "abc"); // "Java" "abc" "Hello"

        // Lấy và hiển thị phần tử của vị trí chỉ định của ArrayList
        System.out.println("3番目の要素" + al.get(2) );

        // Delete phần tử của vị trí chỉ định của ArrayList
        al.remove(2); // "Java" "abc"

        // Lấy và hiển thị số phần tử của ArrayList
        System.out.println("size:" + al.size() );

        // Lấy và hiển thị tất cả phần tử của ArrayList
        for (String str : al) {
            System.out.println(str);
        }
    }
}
```

Mảng có khả năng thay đổi size

ArrayList

[0]	[1]	[2]	[3]	...
-------	-------	-------	-------	-----

Bài tập: ArrayList class

Lưu bội số của 3 trong khoảng từ 1 đến 20 vào ArrayList. Sau khi hiển thị số phần tử của ArrayList, lấy ra từng giá trị đã lưu và hiển thị.

- Class định nghĩa

- Compile/ Kết quả thực hiện

- Gợi ý:

Class name	Folder sử dụng	File name	Chú thích
Ex08_01	labs\Ex08	Ex08_01.java	Creat new

```
C:\%oojava%labs\ex08>java Ex08_01
ArrayList の要素数 (Số phần tử của ArrayList): 6
3
6
9
12
15
18
```

- Trong trường hợp muốn tạo collection coi giá trị của kiểu data cơ bản làm phần tử, chỉ định rapper class cho <パラメータ名> (Parameter name) (nếu là kiểu int thì là Integer class).

HashSet

■ HashSet class

Là class để implement Set interface, thể hiện tập hợp của các giá trị không trùng lặp.

● Ví dụ

```
// Generate/sinh HashSet quản lý data kiểu String
HashSet<String> hs = new HashSet<String>();

// Thêm phần tử cho HashSet
hs.add("Hi");    // "Hi"

// Lấy và hiển thị tất cả phần tử của HashSet
for (String str : hs) {
    System.out.println(str);
}
```

HashSet: Các method chính

Method	Giải thích
boolean add(E o)	Nếu không tồn tại phần tử đã được chỉ định thì add thêm vào và trả về true. Nếu tồn tại thì trả về false
boolean contains(Object o)	Trong trường hợp đang bảo lưu và duy trì phần tử đã được chỉ định thì trả về true.
boolean remove(Object o)	Nếu tồn tại thì xóa phần tử và trả về true. Nếu không tồn tại thì trả về false.
int size()	Trả về số phần tử

HashSet: Ví dụ

```
import java.util.HashSet;

public class HashSet01 {
    public static void main(String[] args) {
        // Generate/sinh HashSet quản lý data kiểu String
        HashSet<String> hs = new HashSet<String>();

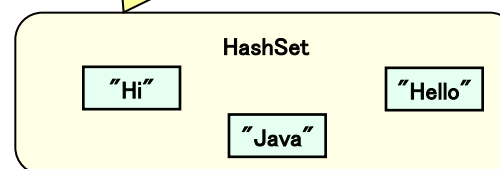
        // Add thêm phần tử cho HashSet
        hs.add("Hi");           // "Hi"
        hs.add("Hello");        // "Hi" "Hello"
        hs.add("Java");          // "Hi" "Hello" "Java"

        // Lấy và hiển thị số phần tử của HashSet
        System.out.println( "size trước khi xóa:" + hs.size() );

        // Delete trong trường hợp tồn tại phần tử đã chỉ định trong HashSet
        if ( hs.remove("Hello") ) {
            System.out.println(Đã xóa "phần tử" "Hello");
        } else {
            System.out.println( "Phần tử" "Hello" " không tồn tại");
        }

        // Lấy và hiển thị tất cả phần tử của HashSet
        for (String str : hs) {
            System.out.println(str);
        }
    }
}
```

Tập hợp của giá trị không trùng lặp



Bài tập: HashSet class

Lưu chuỗi kí tự đã chỉ định khi thực hiện vào HashSet. Sau khi hiển thị số phần tử của HashSet, lấy ra từng giá trị đã lưu và hiển thị (trường hợp đã chỉ thị giá trị bị trùng lặp thì sẽ không được lưu vào HashSet).

- Class định nghĩa

Class name	Folder sử dụng	File name	Chú thích
Ex08_02	C:\ojava\labs\Ex08	Ex08_02.java	Creat new

```
C:\ojava\labs\Ex08>javac Ex08_02.java
```

```
C:\ojava\labs\Ex08>java Ex08_02
```

```
HashSetの要素数 (Số phần tử của HashSet): 0
```

```
C:\ojava\labs\Ex08>java Ex08_02 abc 5 x abc 5
```

```
HashSetの要素数(Số phần tử của HashSet): 3
```

```
abc
```

```
5
```

```
v
```

- HashSet là tập hợp của giá trị không trùng lặp. Dù có tồn tại giá trị đã chỉ định trong add method thì cũng không phát sinh lỗi (trả về false làm giá trị trả về).

- Compile/ Kết quả thực hiện

- Gợi ý:

HashMap

■ HashMap class

Là class để implement Map interface dựa vào hash table, quản lý có kèm xử lý key và giá trị.

● Ví dụ

```
// Generate HashMap quản lý data kiểu String
HashMap<String, String> hm = new HashMap<String, String>();

// Add thêm phần tử vào HashMap
hm.put("e1", "Hi");

// Lấy và hiển thị tất cả phần tử của HashMap (key và giá trị)
for (String str : hm.keySet()) {
    System.out.println(str + ":" + hm.get(str));
}
```

HashMap: Các method chính

Method	Giải thích
V put(K key, V value)	Tạo mối quan hệ của giá trị đã được chỉ định và key đã được chỉ định với map này (giá trị trả về là null). Trường hợp key chỉ định đã tồn tại trong map, thì thay thế giá trị cũ (giá trị trả về là giá trị cũ).
boolean containsKey (Object key)	Trường hợp đã bảo lưu và duy trì key đã được chỉ định thì trả về true
boolean containsValue (Object value)	Trường hợp đang mapping giá trị đã được chỉ định với một hoặc nhiều key thì trả về true.
V remove(Object key)	Nếu tồn tại mapping của key đã được chỉ định thì xóa đi (giá trị trả về là giá trị value đã xóa). Nếu không tồn tại thì trả về null.
int size()	Trả về số phần tử
Set<K> keySet()	Trả về Set của key có trong map này.

HashMap: Ví dụ

```
import java.util.HashMap;

public class HashMap01 {
    public static void main(String[] args) {
        // Generate HashMap quản lý data kiểu String
        HashMap<String, String> hm = new HashMap<String, String>();

        // Add phần tử cho HashMap
        hm.put("e1", "Hi");
        hm.put("e2", "Hello");
        hm.put("e3", "Java");

        // Lấy và hiển thị số phần tử của HashMap
        System.out.println( "size trước khi xóa:" + hm.size() );

        // Delete trong trường hợp đã tồn tại phần tử đã chỉ định trong HashMap
        if ( hm.remove("e2") != null ) {
            System.out.println(Đã ~ xóa "phần tử" e2~");
        } else {
            System.out.println( "phần tử" e2~ " không tồn tại");
        }

        // Lấy và hiển thị tất cả các phần tử của HashMap (key và giá trị)
        for (String str : hm.keySet()) {
            System.out.println(str + ":" + hm.get(str));
        }
    }
}
```

Tập hợp của set key và giá trị

HashMap

"e1"	"e2"	"e3"	"e4"
"Hi"	"Java"	"Hello"	"ABC"

Generics

- Generics là chức năng có kèm xử lý kiểu đặc định khi sử dụng những phần đã được định nghĩa như là class và method thông dụng.
- Cú pháp:

```
Modifier class class name <parameter name> {  
    :  
}
```
- Sau khi thực hiện bảo đảm kiểu khi compile, có thể sử dụng kiểu và method đối với nhiều kiểu đa dạng.

```
Class name<kiểu đặc định> Tên biến = new classname<kiểu đặc định>(List đối số);
```

Nêu vấn đề

- Trước khi có Generics:

```
List list = new ArrayList( );
```

```
list.add("Hello");
```

```
String s = (String) list.get(0); // Phải cast về String
```

- Sau khi có Generics:

```
List<String> list = new ArrayList<>( );
```

```
list.add("Hello");
```

```
String s = list.get(0); // Không cần cast, an toàn kiểu dữ  
liệu
```

Generics: Ưu điểm

- **Generics** cho phép viết code tổng quát mà vẫn đảm bảo **an toàn kiểu dữ liệu** (type safety).
- Bạn có thể định nghĩa class, method hoạt động với nhiều kiểu dữ liệu khác nhau mà không cần ép kiểu (casting) thủ công.

Generics: Ưu điểm (2)

- Generics giải quyết vấn đề:
 - Tránh lỗi runtime do cast sai kiểu.
 - Giúp code rõ ràng, dễ đọc, dễ bảo trì.
 - Tái sử dụng class và method cho nhiều kiểu dữ liệu khác nhau.

Generic Class

- Generic class là class được định nghĩa với tham số kiểu (type parameter), giúp class làm việc với nhiều kiểu dữ liệu khác nhau mà vẫn đảm bảo an toàn kiểu dữ liệu (type safety).

Tại sao cần Generic Class?

- Trước Generics, bạn phải dùng Object cho mọi kiểu dữ liệu → cần casting thủ công khi lấy ra, dễ gây lỗi runtime.

Với Generic Class, compiler sẽ kiểm tra kiểu dữ liệu ngay từ compile time.

Ví dụ:

```
class InventoryItem<T> {  
    private T id;  
    private String name;  
  
    public InventoryItem(T id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public T getId() { return id; }  
    public String getName() { return name; }  
}
```

```
InventoryItem<Integer> item1 = new InventoryItem<>(101, "Laptop");  
InventoryItem<String> item2 = new InventoryItem<>("SKU555", "iPhone");
```

Generic Method

- Generic method là method khai báo tham số kiểu riêng trong chính method đó.
- Nó hoạt động độc lập, không phụ thuộc class có generic hay không.

Ví dụ:

```
public static <T> void printArray(T[ ] arr) {  
    for (T e : arr) {  
        System.out.print(e + " ");  
    }  
    System.out.println();  
}  
printArray(names); // Alice Bob  
printArray(nums); // 1 2 3
```